Experiment – 8

Name: Diwansh
UID: 24BAI70294
Section: 24AIT_KRG-G1

Step 1: Implementation

- **Binary Search (Divide and Conquer):**

Binary Search is an efficient searching technique that works only on a sorted array. The algorithm begins by initializing two pointers, commonly called low and high, which represent the current search boundaries. It repeatedly calculates the middle index and compares the target value with the element at that position. If the target matches, the search ends successfully. Otherwise, the algorithm eliminates half of the remaining search space—either the left half or the right half—depending on whether the target is smaller or larger than the middle element. This process continues iteratively until the element is found or the search space becomes empty, making it extremely efficient for large datasets.

- **Subset Sum – Decision Version (Brute Force):**

The decision version of the Subset Sum problem involves determining whether any subset of a given set adds up to a specific target value. The brute-force approach systematically explores all possible subsets using recursion. At each step, the algorithm makes a binary decision: include the current element in the subset or exclude it. This creates a decision tree with multiple branches, representing all possible combinations. The algorithm checks each path to see if the sum equals the target. Due to this exhaustive exploration, the approach becomes computationally expensive as the input size increases.

- **Subset Sum – Verification Version:**

The verification version is much simpler compared to the decision version. Instead of searching for a solution, it assumes a proposed subset is already given. The algorithm only needs to validate whether this subset satisfies the required condition. It iterates through the subset, computes the sum of its elements, and compares it with the target value. Since it performs only a single pass through the subset, the process is straightforward, fast, and efficient.

- **Traveling Salesman Problem (Brute Force):**

The brute-force solution to the Traveling Salesman Problem (TSP) involves checking every possible route that visits all cities exactly once and returns to the starting city. This is done by generating all permutations of the cities (excluding the starting city to avoid redundant cycles). For each permutation, the algorithm calculates the total travel cost by summing the distances between consecutive cities and adding the return distance. It keeps track of the minimum cost among all possible routes. Although this guarantees the optimal solution, the approach is extremely slow due to the rapid growth in the number of permutations.

Step 2: Measuring Metrics

When evaluating these algorithms based on performance metrics such as execution time, number of operations, and feasibility, the following observations can be made:

- **Binary Search:**

The number of operations grows very slowly even as the input size increases significantly. Because the search space is halved at every step, the algorithm performs only a small number of comparisons. This results in extremely fast execution times, often appearing instantaneous, and it consistently completes successfully even for very large datasets.

- **Subset Sum (Decision):**

The number of operations increases exponentially with each additional element in the input set. Since every element introduces two possibilities (include or exclude),

the total combinations double each time. This leads to a sharp rise in execution time, and the algorithm quickly becomes impractical for larger inputs, often resulting in timeouts beyond a certain size.

- **Subset Sum (Verification):**

The operations depend directly on the size of the given subset. Since it only involves summing the elements once, the execution time remains minimal. The algorithm is highly efficient and always completes successfully regardless of input size.

- **TSP (Brute Force):**

The number of operations grows factorially, which is even faster than exponential growth. As the number of cities increases, the number of possible routes becomes enormous. This causes execution time to skyrocket, making the algorithm infeasible for even moderately large inputs.

Part B: Analysis Tasks (Critical Thinking)

1. Why is Binary Search so efficient?

Binary Search is efficient because it follows the divide and conquer strategy, where the problem size is reduced by half after every comparison. Instead of checking each element sequentially, it directly targets the middle element and decides which half of the array can be ignored. This drastically reduces the number of operations required.

The time complexity of Binary Search is O(log n), which means even if the input size grows very large, the increase in operations is very small. This makes it highly scalable and suitable for large datasets, provided the data is sorted.

Example:
Array = [2, 5, 8, 12, 16, 23], target = 12

- **Step 1: Middle = 8 → target is greater**

- **Step 2: Search right half → middle = 12 → found**

Only a few steps are needed, showing its efficiency.

---

2. Why is Subset Sum hard to solve but easy to verify?

The Subset Sum problem highlights the difference between searching for a solution and verifying a solution. To solve it, the algorithm must consider all possible subsets of the given set. Since each element has two choices (included or excluded), the total number of subsets becomes 2^n, leading to exponential growth.

However, verifying a given subset is much simpler. It only requires summing the elements and comparing the result to the target. This can be done in linear time, making verification efficient.

This distinction is fundamental in complexity theory and is the reason why Subset Sum belongs to the NP (Nondeterministic Polynomial time) class.

Example:
Set = {2, 4, 6}, target = 6

- **Solving: Check {2}, {4}, {6}, {2,4}, etc.**
- **Verifying: {2,4} → 2 + 4 = 6 → correct**

---

3. When does TSP become infeasible?

The Traveling Salesman Problem becomes infeasible due to its factorial time complexity O(n!), which grows extremely fast compared to polynomial or even exponential growth. In the brute-force approach, every possible permutation of cities must be checked to find the shortest route.

As the number of cities increases, the number of possible routes increases drastically, making computation impractical even for powerful systems. This is why approximate or heuristic algorithms are often used in real-world applications instead of brute force.

Example:
For 3 cities → 2 routes (easy)
For 5 cities → 120 routes
For 10 cities → 36,28,800 routes

Beyond 12–15 cities, it becomes infeasible to compute within a reasonable time.

---

4. Solving vs Verifying a Problem

In computational problems, there is an important distinction between solving (finding a solution) and verifying (checking a given solution).

- **Solving: Requires exploring multiple possibilities, applying logic, and computing the correct answer from scratch. This can be time-consuming depending on the problem complexity.**
- **Verifying: Requires checking whether a provided solution satisfies the conditions of the problem. This is usually much faster and involves straightforward calculations.**

This concept is central to understanding complexity classes like P and NP, where problems in NP are easy to verify but not necessarily easy to solve.

Example:
Set = {1, 3, 5}, target = 8

- **Solving: Try combinations → find {3,5}**
- **Verifying: {3,5} → 3 + 5 = 8 → correct**

---

5. Why are NP-Complete problems the hardest?

NP-Complete problems are considered the most challenging because they represent the hardest problems within the NP class. A problem is NP-Complete if:

1. **A proposed solution can be verified in polynomial time (belongs to NP)**
2. **Every other NP problem can be reduced to it in polynomial time**

This means NP-Complete problems act as a benchmark for computational difficulty. If an efficient (polynomial-time) solution is discovered for even one NP-Complete problem, it would imply that all NP problems can also be solved efficiently.

Because of this, NP-Complete problems are widely studied in computer science, especially in optimization, cryptography, and algorithm design.

Example:
Subset Sum

- **Finding subset → difficult (exponential time)**
- **Checking subset {3,5} for target 8 → easy**